FairClaim

Technical Architecture & Build Specification

Build with Gemini XPRIZE · Companion to the Execution & Business Plan

Prepared July 1, 2026 · Build window: July 4–14 · Solo developer

1. Stack Overview

The stack is chosen for three properties: it must be shippable by one person in eleven days, it must run at near-zero marginal cost, and it must be visibly built on Google's AI stack, since demonstrating Gemini-native operations is a judged criterion. Boring technology everywhere except the agent pipeline, which is where the differentiation lives.

Layer	Choice	Rationale
Frontend	Next.js (single app, server-rendered)	One codebase for landing pages, intake flow, and order status; vertical/state landing pages are just routes, which matters for the GTM plan.
Backend	Same Next.js app (API routes) + a worker for the pipeline	No microservices. One deployable unit plus one queue worker.
AI	Gemini API via AI Studio keys; Flash tier for intake/QA/extraction, Pro tier for drafting	Flash handles high-volume structured steps at trivial cost; Pro reserved for the one step where writing quality is the product.
Hosting	Cloud Run (app + worker), Cloud Tasks for the order queue	Scales to zero, covered by the $300 competition credits, and is part of the Google stack story.
Data	Firestore (orders, customers, letters) + Cloud Storage (uploads, PDFs, 30-day lifecycle rule)	Serverless, zero ops, native TTL-style cleanup via lifecycle rules satisfies the 30-day deletion promise.
Payments	Stripe Checkout + webhooks	Fastest path to charging; the dashboard doubles as judge-facing revenue evidence.
Email	Transactional email service (e.g., Resend/Postmark)	Order delivery, day-7/day-21 outcome follow-ups.
PDF	Server-side HTML-to-PDF rendering	Letters are HTML templates; same template renders web preview and PDF.

2. System Architecture

The system is a thin storefront wrapped around an asynchronous agent pipeline. Nothing in the request path does AI work; payment confirmation enqueues an order, the worker runs the pipeline, and the customer is notified when the letter is ready (target: under ten minutes, typical: under three).

```
Customer → Intake form (structured fields + document upload)
  → Stripe Checkout → payment webhook → order created (Firestore) → enqueued (Cloud Tasks)
  → Worker (Cloud Run): Agent pipeline stages 1–5 (Section 3)
  → PDF rendered → stored (Cloud Storage) → delivery email → order page updated
  → Day-7 / day-21 outcome follow-up emails (scheduled tasks)
```

Failure handling is deliberately simple: any pipeline stage that fails validation retries once with the validator's critique appended; a second failure parks the order in a review queue and notifies you. During week one of automation, every order routes through the review queue regardless (approve-and-send, one click), which is how quality gets verified without becoming the bottleneck.

3. The Agent Pipeline

Five stages, each a separate Gemini call with a narrow contract and structured JSON output. Separation is what makes quality controllable: each stage can be tested, logged, and improved independently, and the pipeline logs become the AI-native-operations evidence for the submission.

Stage	Model	Contract
1. Intake validation & extraction	Flash	Input: raw intake fields + uploaded document text (extracted via Gemini multimodal). Output: normalized case file JSON — parties, dates, amounts, dispute class, jurisdiction — plus a completeness check that emails the customer for anything critical that's missing (e.g., move-out date in a deposit case).
2. Jurisdiction research	Flash + curated reference set	Selects applicable statutes from a hand-curated YAML reference library (Section 4) keyed by state and dispute class. The model chooses and applies from the library; it never free-recalls citations. Output: statute list with deadlines, penalty multipliers, and required-notice rules for this case.
3. Drafting	Pro	Writes the demand letter from the case file + statute set: formal-but-firm tone, first person in the customer's voice, chronology, statutory basis, specific demand amount with computation shown, response deadline, consequences paragraph. Also drafts the plain-English cover note.
4. Adversarial QA	Flash	A separate reviewer prompt attacks the draft: verifies every citation exists in the reference set and matches the case facts, checks arithmetic, checks dates, flags any sentence that could read as FairClaim (rather than the customer) giving legal advice. Output: pass, or a critique fed back to stage 3 for one retry.
5. Escalation guidance (paid tier)	Flash	Assembles the small-claims roadmap from a per-state procedural reference (filing venue, fee ranges, service rules), the follow-up letter template, and the evidence checklist.

Every stage logs input hash, output, model, latency, and token cost to Firestore. A tiny internal dashboard (orders/day, median delivery time, QA failure rate, cost per order) is worth the half-day it takes: it is simultaneously your ops tool and a screenshot for the submission.

4. The Statute Reference Library

This is the real moat of the product and the main defense against the embarrassing-citation risk. It is a curated, human-verified YAML dataset — not model knowledge — covering, per state and dispute class: statute citation, plain-language summary, deadline rules, penalty provisions, and any required pre-suit notice. Build order: start with the 10 most populous states for the security-deposit vertical (roughly 55% of the US population), add the remaining verticals for those states, then broaden state coverage as orders arrive from elsewhere. Gemini can draft each entry from the statute text; you verify against the actual state code before it enters the library. Budget: ~2 days across the build window, then ~30 minutes per new state on demand. Cases from uncovered states fall back to a general-principles letter (still effective) with the customer told plainly that no state statute is cited.

5. Data Model

```
customers: id, email, created_at
orders: id, customer_id, tier, status (intake→paid→processing→review→delivered),
dispute_class, state, amounts, stripe_ids, timestamps
case_files: order_id, normalized JSON from stage 1, statute set from stage 2
letters: order_id, draft versions, QA verdicts, final PDF path, revision_count
pipeline_runs: order_id, stage, model, latency_ms, tokens, cost, pass/fail
outcomes: order_id, day7_response, day21_response, recovered_amount, testimonial_permission
```

The outcomes table is small but strategically loaded: it feeds the total-recovered counter, the testimonial engine, and the impact section of the submission. Wire the day-7/day-21 emails to write into it via one-click links ("They paid / They responded / Nothing yet") so response friction is near zero.

6. Build Order (July 4–14)

Days	Deliverable
Jul 4–5	Skeleton: Next.js app deployed to Cloud Run from day one; Firestore + Storage wired; Stripe Checkout for the two tiers; intake form storing a complete case record. (Manual fulfillment continues off this intake data.)
Jul 6–8	Pipeline stages 1–3 working end-to-end on real orders from Phase 0; letter HTML template + PDF rendering; delivery email. Statute library: deposit vertical, top 10 states.
Jul 9–10	Stage 4 QA agent + retry loop + review queue with one-click approve; pipeline logging; internal metrics dashboard.
Jul 11–12	Stage 5 escalation pack; response-review add-on flow; day-7/21 outcome emails and outcomes table; remaining verticals in the statute library for top 10 states.
Jul 13–14	Hardening: error paths, refund flow, disclaimers everywhere per the legal positioning, 30-day storage lifecycle rule, privacy page, 2 vertical landing pages. Production cutover with 100% review-queue routing.

Scope discipline: anything not in this table after July 14 requires a demonstrated sales blocker to justify. The plan's base-case revenue scenario depends on ~30 uninterrupted days of distribution work.

7. Prompt Engineering Guidelines

Keep every stage's prompt in version control with a changelog; when a letter needs manual correction, the fix goes into the prompt or the reference library, never just the individual letter. Use structured output (JSON schema) for stages 1, 2, 4, and 5; stage 3 outputs markdown against a strict letter skeleton. Maintain a small golden set of ~15 real anonymized cases with approved letters; rerun the pipeline against it after any prompt change and diff the results before deploying. The Phase 0 manual orders are the seed of this golden set — save everything from day one. Two content rules enforced in the drafting prompt and checked by QA: the letter never asserts legal conclusions beyond what the cited statute plainly provides, and every dollar figure must trace to an intake field or a statutory multiplier with the computation shown.

8. Quality, Safety, and Trust Guardrails

Beyond the QA stage: intake screens out disputes the product must not touch (anything criminal, family law, active litigation, injury claims, or amounts above small-claims jurisdiction get a polite refusal and refund rather than a letter); rate limiting and disposable-email checks deter abuse of the free-revision policy; all customer documents are encrypted at rest by default on Google Cloud, deleted at 30 days by lifecycle rule, and never leave the order context; prompts include no other customer's data. The "not a law firm / not legal advice / you are responsible for reviewing before sending" disclaimer appears at intake, on the cover note, in the delivery email, and in the footer.

9. Deployment, Operations, and Cost

Deployment is a single Cloud Run service plus a worker, deployed from GitHub via Cloud Build on every merge; staging is unnecessary at this scale — the review queue is the safety net. Monitoring is the internal dashboard plus an email alert on any parked order. Estimated costs at the base-case volume (100 orders): Gemini across all stages roughly $0.10–0.30 per order at 2026 Flash/Pro pricing, Cloud Run and Firestore effectively free at this traffic, Stripe ~$1.43 per $39 order — total marginal cost under $2, against the $300 Cloud + $100 AI credits granted at registration. Verify current model pricing and credit terms in AI Studio when registering, and keep the billing export enabled: a real cost-per-order number is a strong line in the business-viability section of the submission.